



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 内核线程

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/3/30-2026/4/10

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
五、实验总结与心得体会.....	15
六、附录：参考资料清单.....	16

Section 1 实验概述

在本次实验将会学习到 C 语言的可变参数机制的实现方法。在此基础上，会揭开可变参数背后的原理，进而实现可变参数机制。实现了可变参数机制后，我们将实现一个较为简单的 `printf` 函数。

本次实验另外一个重点是内核线程的实现，我们首先会定义线程控制块的数据结构——PCB。然后，我们会创建 PCB，在 PCB 中放入线程执行所需的参数。最后，我们会实现基于时钟中断的时间片轮转(RR)调度算法。在这一部分中，我们需要重点理解 `asm_switch_thread` 是如何实现线程切换的，体会操作系统实现并发执行的原理。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++/C 程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）/vscode
 - 代码编译工具：GCC、g++、nasm、ld（链接器）、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：改进 `printf` 函数
- 实验任务 2：设计 PCB
- 实验任务 3：os 线程的并发执行
- 实验任务 4：修改线程调度算法

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：学习可变参数机制，然后实现 `printf` 函数，可以在材料中（src/3）的 `printf` 上进行改进，或者从头开始实现 `printf` 函数。
- 思路分析：
 - 1.如若后缀错误.cpp->.h，由于 makefile 已经确定，会导致屏幕一直弹射变换，最终编译失败

```

../src/kernel/stdio.h:165:5: note: 'int printf(const char*, ...)' previously defined here int
printf(const char *const fmt, ...) ^~~~~~ In file included
from ../src/kernel/interrupt.cpp:5:0: ../src/kernel/stdio.h:206:28: error: 'stdio' was not
declared in this scope counter += stdio.print(buffer);
^~~~~ ../src/kernel/stdio.h:206:28: note: suggested alternative: 'static' counter +=
stdio.print(buffer); ^~~~~ static ../src/kernel/stdio.h:232:16: error: 'stdio' was not
declared in this scope counter += stdio.print(buffer);
^~~~~ ../src/kernel/stdio.h:232:16: note: suggested alternative: 'static' counter +=
stdio.print(buffer); ^~~~~ static ../src/kernel/interrupt.cpp: At global
scope: ../src/kernel/interrupt.cpp:8:8: error: 'STDIO' does not name a type; did you mean
'STDLIB_H'? extern STDIO stdio; ^~~~~ STDLIB_H ../src/kernel/interrupt.cpp: In
function 'void c_time_interrupt_handler()': ../src/kernel/interrupt.cpp:94:9: error: 'stdio'
was not declared in this scope stdio.print(0, i, ' ', 0x07);
^~~~~ ../src/kernel/interrupt.cpp:94:9: note: suggested alternative: 'static'
stdio.print(0, i, ' ', 0x07); ^~~~~ static ../src/kernel/interrupt.cpp:114:5: error: 'stdio' was
not declared in this scope stdio.moveCursor(0); ^~~~~ makefile:49: recipe for target
'setup.o' failed make: *** [setup.o] Error 1

```

2. 选择在 src/3 的 printf 的基础上加入八进制和浮点数的打印

● 实验步骤:

①修改 makefile: 示例 makefile 与以下命令代码结合, 防止磁盘被“污染”导致没有输出

```

cd project1/run
rm hd.img
dd if=/dev/zero of=hd.img bs=1M count=10
cd ../build
make clean
make build
qemu-system-i386 -hda ../run/hd.img -vga std

```

②改进 printf 代码

▲打印字符串添加空指针保护

```

1 case 's': {
2     buffer[idx] = '\0';
3     counter += stdio.print(buffer);
4     idx = 0;
5     const char *s = va_arg(ap, const char *);
6     counter += stdio.print(s ? s : "(null)"); // 增加判空保护
7     break;
8 }

```

▲增加八进制打印:

其中，直接在处理循环中完成进制转换，省去 `itos` 和 `swap` 的中间开销
并且使用逆序压入缓冲区能够避免在 `swap` 中多次扫描字符串

```
1  case 'd':
2      case 'x':
3      case 'o': { // 新增八进制
4          int val = va_arg(ap, int);
5          unsigned int u_val;
6          int base;
7
8          if (fmt[i] == 'd') base = 10;
9          else if (fmt[i] == 'x') base = 16;
10         else base = 8;
11
12         // 处理负数
13         if (fmt[i] == 'd' && val < 0) {
14             counter += printf_add_to_buffer(buffer, '-', idx, BUF_LEN);
15             u_val = (unsigned int)(-val);
16         } else {
17             u_val = (unsigned int)val;
18         }
19
20         // 原地转换数字，无需调用额外的 itos 和 swap
21         int pos = 0;
22         do {
23             unsigned int remainder = u_val % base;
24             number[pos++] = (remainder > 9) ? (remainder - 10 + 'A') : (remainder + '0');
25             u_val /= base;
26         } while (u_val > 0);
27
28         // 逆序压入缓冲区
29         while (pos > 0) {
30             counter += printf_add_to_buffer(buffer, number[--pos], idx, BUF_LEN);
31         }
32         break;
```

▲增加浮点数打印:

实现 `printf` 中 `%f` 格式（浮点数）的打印功能:

`double` 浮点数先判断是否为负数，若是则先向打印缓冲区写入负号，并将浮点数转为正数以便统一处理;

接着将浮点数拆分为整数部分和小数部分

然后采用“取余取低位、除 10 删低位”的逆序转换方法，将整数部分转为字符存入临时缓冲区，再逆序写入打印缓冲区完成整数部分的打印;

之后向缓冲区写入小数点

再以同样的逆序转换方法处理小数部分，若小数部分不足 6 位，则自动补 0 至 6 位

```

1 case 'f': { // 浮点数
2     double f_val = va_arg(ap, double);
3     char f_buf[32];
4     int f_pos = 0;
5
6     // 处理负数
7     if (f_val < 0) {
8         counter += printf_add_to_buffer(buffer, '-', idx, BUF_LEN);
9         f_val = -f_val;
10    }
11
12    // 拆分整数 + 小数
13    unsigned int int_part = (unsigned int)f_val;
14    unsigned int dec_part = (unsigned int)((f_val - int_part) * 1000000); // 6位小数
15
16    // 打印整数部分
17    do {
18        f_buf[f_pos++] = (int_part % 10) + '0';
19        int_part /= 10;
20    } while (int_part > 0);
21    while (f_pos > 0) {
22        counter += printf_add_to_buffer(buffer, f_buf[--f_pos], idx, BUF_LEN);
23    }
24
25    // 小数点
26    counter += printf_add_to_buffer(buffer, '.', idx, BUF_LEN);
27
28    // 打印小数部分 (补0到6位)
29    f_pos = 0;
30    do {
31        f_buf[f_pos++] = (dec_part % 10) + '0';
32        dec_part /= 10;
33    } while (dec_part > 0);
34    for (int j = 0; j < 6 - f_pos; j++) counter += printf_add_to_buffer(buffer, '0', idx, BUF_LEN);
35    while (f_pos > 0) {
36        counter += printf_add_to_buffer(buffer, f_buf[--f_pos], idx, BUF_LEN);
37    }
38    break;
39 }

```

③修改 setup.cpp 代码进行测试

```

1 printf("print percentage: %%\n"
2     "print char \"N\": %c\n"
3     "print string \"Hello World!\": %s\n"
4     "print decimal: \"-1234\": %d\n"
5     "print hexadecimal \"0x7abcdef0\": %x\n"
6     "print octal \"1234\": %o\n"           // 八进制
7     "print float \"3.1415926\": %f\n"     // 浮点数
8     "print float \"-123.456\": %f\n",     // 负浮点数
9     'N', "Hello World!", -1234, 0x7abcdef0, 1234, 3.1415926, -123.456);

```

- 实验结果：点击 make 运行，能够实现%，字符，字符串，十进制，十六进制，八进制以及浮点数的打印

```

Booting from Hard Disk...
print percentage: %
print char "N": N
print string "Hello World!": Hello World!
print decimal: "-1234": -1234
print hexadecimal "0x7abcdef0": 7ABCDEF0
print octal "1234": 2322
print float "3.1415926": 3.141592
print float "-123.456": -123.456000

lalala@lalala-VirtualBox: ~/lab4/src/project/build$ make

```

实验任务 2

- 任务要求：自行设计 PCB，可以添加更多的属性，如优先级等，然后根据新的 PCB 来实现线程，演示执行结果。

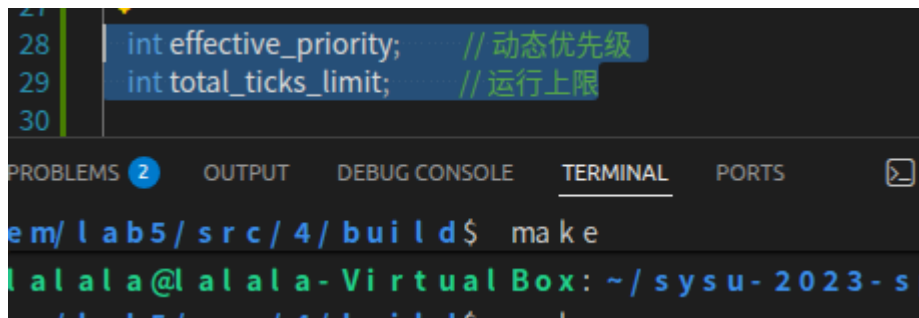
- 思路分析：

1. 规定线程只能执行返回值为 void，参数为 void *的函数，其中，void *指向了函数的参数

2. 关中断，实现线程互斥

- 实验步骤：

①给 PCB 增加新成员——运行限制和动态优先级



```
28 int effective_priority; // 动态优先级
29 int total_ticks_limit; // 运行上限
30
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

em/ lab5/ src/ 4/ build\$ make

lalala@lalala-Virtual Box: ~/sysu-2023-s

②在 src/kernel/program.cpp 中的 executeThread 中给新属性赋初值



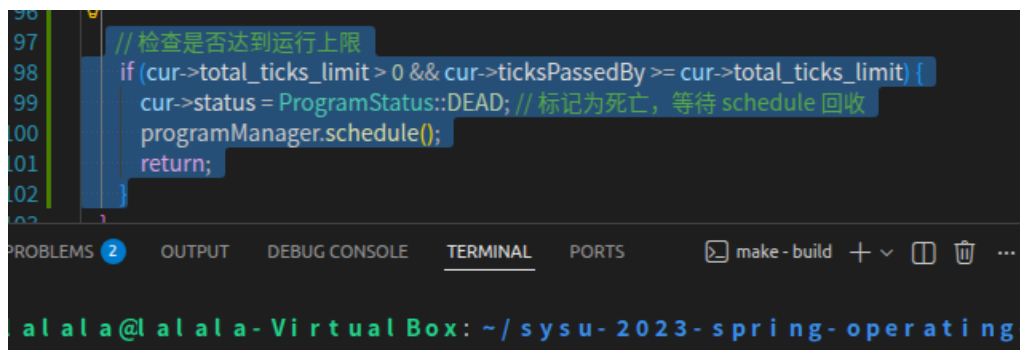
```
54 // ---
55 thread->effective_priority = priority; // 初始动态优先级等于静态优先级
56 thread->total_ticks_limit = 500; // 假设默认最多运行 500 个时钟滴答，演示自动退出逻辑
57 // -----
58 thread_spid = ((int)thread_((int)PCB_SET/PCB_SIZE,
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

make-build + v [] [] ...

lalala@lalala-Virtual Box: ~/sysu-2023-spring-operating-

③修改运行时监控逻辑（c_time_interrupt_handler）：运行限制的运用



```
97 // 检查是否达到运行上限
98 if (cur->total_ticks_limit > 0 && cur->ticksPassedBy >= cur->total_ticks_limit) {
99     cur->status = ProgramStatus::DEAD; // 标记为死亡，等待 schedule 回收
100     programManager.schedule();
101     return;
102 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

make-build + v [] [] ...

lalala@lalala-Virtual Box: ~/sysu-2023-spring-operating-

④修改调度算法，采用动态优先级调整（schedule）：动态优先级运用

```

if (running->status == ProgramStatus::RUNNING)
{
    running->status = ProgramStatus::READY;
    // 动态优先级逻辑:
    // 如果线程是因为用光了时间片才被调度, 说明它很“贪婪”, 降低其动态优先级
    if (running->effective_priority > 1) {
        running->effective_priority -= 1;
    }

    // 根据新的动态优先级分配下一个时间片
    running->ticks = running->effective_priority * 10;
    readyPrograms.push_back(&(running->tagInGeneralList));
}
else if (running->status == ProgramStatus::DEAD)
{
    releasePCB(running);
}

```

- 实验结果展示:

```

SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
pid 0 name "first thread": Hello World!
-

lalala@lalala-VirtualBox: ~/sysu-2023-spring-operating-syst
em/lab5/src/4/build$ make run
ted.

```

----- 实验任务 3 -----

- 任务要求: os 的线程能够并发执行——编写若干个线程函数, 使用 gdb 跟踪 `c_time_interrupt_handler`、`asm_switch_thread` (eg: `b c_time_interrupt_handler`) 等函数, 观察线程切换前后栈、寄存器、PC 等变化, 结合 gdb、材料中“线程的调度”的内容来跟踪并说明下面两个过程。1. 一个新创建的线程是如何被调度然后开始执行的。2. 一个正在执行的线程是如何被中断然后被换下处理器的, 以及换上处理机后又是如何从被中断点开始执行的。进一步理解操作系统是如何实现线程的并发执行的。

- 实验步骤:

- ①修改 setup.cpp

```
void third_thread(void *arg) {
    // PID
    printf("pid%d", programManager.running->pid);
    for(int i = 0; i < 1000000; ++i); // 延时
}

void second_thread(void *arg) {
    printf("pid%d,24325242txc", programManager.running->pid);
    for(int i = 0; i < 1000000; ++i);
}

void first_thread(void *arg)
{
    printf("pid %d name \"%s\": Hello World!\n",
        programManager.running->pid, programManager.running->name);

    // 在第一个线程中创建其他线程
    programManager.executeThread(second_thread, nullptr, "second", 1);
    programManager.executeThread(third_thread, nullptr, "third", 1);

    while(1) {
        asm_halt();
    }
}
```

MS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + v

la@l al a l a - V i r t u a l B o x : ~ / s y s u - 2 0 2 3 - s p r i n g - o p e r

Make debug 进行 GDB 调试

- ②设置断点并运行到断点部分

```

Booting from Hard Disk...

终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/utls/asm_utils.asm
19     ASM_IDTR dw 0
20         dd 0
21
22     ; void asm_switch_thread(PCB *cur, PCB *next);
23     asm_switch_thread:
B+> 24     push ebp
25         push ebx
26         push edi
27         push esi
28
29         mov eax, [esp + 5 * 4]
30         mov [eax], esp ; 保存当前栈指针到PCB中, 以便日后
31

remote Thread 1 In: asm_switch_thread          L24   PC: 0
0x0000ffff in ?? ()
(gdb) b asm_switch_thread
Breakpoint 1 at 0x214ec: file ../src/utls/asm_utils.asm, line 24.
(gdb) c

```

②确定当前是切换到新进程

▲eax 存放的是 next 指针:

```

32     mov eax, [esp + 6 * 4]
> 33     mov esp, [eax] ; 此时栈已经从cur栈切换到next栈
34
35     pop esi
36     pop edi
37     pop ebx

remote Thread 1 In: asm_switch_thread          L33   PC: 0x214
eax          0x0      0
(gdb) s
Undefined command: "s". Try "help".
(gdb) s
(gdb) s
(gdb) info reg eax
eax          0x21d60   138592
(gdb)

```

```

(gdb) p (struct PCB *)$eax
$1 = (PCB *) 0x21d60 <PCB_SET>

```

▲ticksPassedBy 为 0, 说明它是第一次运行

```

(gdb) p ((struct PCB *)$eax)->ticksPassedBy
$2 = 0

```

③观察栈顶数据

▲旧栈顶

```

(gdb) x/12xw $esp
0x7bc0: 0x00000000      0x00000000      0x00039000      0x00007bfc
0x7bd0: 0x00020648      0x00000000      0x00021d60      0x00000000
0x7be0: 0x00000080      0x00000001      0x00021d94      0x00021d60

```

▲新栈顶：此时栈顶内容：

- [0, 0, 0, 0]（对应 pop 进 ebp, ebx, edi, esi）
- 函数地址
- program_exit 的地址

```
(gdb) x/10xw $esp
0x22d44 <PCB_SET+4068>: 0x00000000      0x00000000      0x00000000      0x00000000
00
0x22d54 <PCB_SET+4084>: 0x000204fb      0x000203b8      0x00000000      0x00000000
00
0x22d64 <PCB_SET+4100>: 0x00000000      0x00000000
(gdb)
```

④新创建线程开始执行：经过 4 个 pop 指令后，执行完 ret 后 EIP 变成了线程函数的首地址

```
(gdb) i r eip
eip      0x204fb  0x204fb <first_thread(void*)>
```

⑤捕获时间中断

```
88     extern "C" void c_time_interrupt_handler()
89     {
B+> 90         PCB *cur = programManager.running;
91
92         if (cur->ticks)
93         {
94             --cur->ticks;
95             ++cur->ticksPassedBy;
96
97             // 检查是否达到运行上限

remote Thread 1 In: c time interrupt handler      L90    PC: 0x208fa
eip      0x204fb  0x204fb <first_thread(void*)>
(gdb) b c_time_interrupt_handler
Breakpoint 2 at 0x208fa: file ../src/kernel/interrupt.cpp, line 90.
(gdb) c
Continuing.
```

⑥某个线程的函数被中断

```
Breakpoint 2, c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:90
(gdb) bt
#0  c_time_interrupt_handler () at ../src/kernel/interrupt.cpp:90
#1  0x0002151c in asm_time_interrupt_handler ()
    at ../src/utils/asm_utils.asm:67
#2  0x00000000 in ?? ()
```

⑦跟踪换下过程：在执行 mov[ecx], esp 后，发现 PCB 记录了此时的栈顶，现场已保存

```

23     asm switch_thread:
B+> 24     push ebp
25     push ebx
26     push edi
27     push esi
28
29     mov eax, [esp + 5 * 4]
30     mov [eax], esp ; 保存当前栈指针到PCB
31
remote Thread 1 In: asm_switch_thread
Breakpoint 1 at 0x214ec: file ../src/utils/asm_utils
(gdb) c
Continuing.

Breakpoint 1, asm_switch_thread () at ../src/utils/a
(gdb) i r esp
esp             0x7bd0    0x7bd0

```

```

29     mov eax, [esp + 5 * 4]
30     mov [eax], esp ; 保存当前栈指针到PCB中，以便日后
31
32     mov eax, [esp + 6 * 4]
33     mov esp, [eax] ; 此时栈已经从cur栈切换到next栈
34
35     pop esi
36     pop edi
37     pop ebx

```

```

ote Thread 1 In: asm_switch_thread                                L32    PC: 0
b) s
b) s
b) s
b) s
b) s
b) p/x ((struct PCB *)$eax)->stack
= 0x7bc0
b) 

```

⑧执行 `mov esp, [eax]`，ESP 回到了上次保存的位置

```

32      mov eax, [esp + 6 * 4]
33      mov esp, [eax] ; 此时栈已经从cur栈切换到next栈
34
> 35      pop esi
36      pop edi
37      pop ebx
38      pop ebp
39
40      sti

:remote Thread 1 In: asm_switch_thread                                L35  P
(db) p/x ((struct PCB *)$eax)->stack
= 0x7bc0
(db) s
(db) s
asm_switch_thread () at ../src/utls/asm_utils.asm:35
(db) i r esp
p      0x22d44  0x22d44 <PCB_SET+4068>

```

⑨回到被中断点：时钟中断处理结束，`eip` 回到了最初被中断的那行代码

```

(gdb) i r eip
eip      0x204fb  0x204fb <first_thread(void*)>

```

● 实验结果展示：

▲代码输出

```

Booting from Hard Disk...
pid 0 name "first thread": Hello World!
pid1,24325242txcpid2

```

▲问题回答：

1. 一个新创建的线程是如何被调度然后开始执行的？

伪造栈帧：在线程创建阶段（`executeThread`），操作系统在 `PCB` 指向的内存页高地址处手动压入了初始数据。这个栈从上到下依次是：线程参数、`program_exit` 返回地址、线程函数入口地址，以及 4 个寄存器占位符（0）。

- **切换栈指针：**当调度器选择该线程并进入 `asm_switch_thread` 后，执行 `mov esp, [eax]` 指令。此时，CPU 的 `ESP`（栈指针）瞬间从旧线程的栈跳变到了这个伪造的栈上。
- **利用 `ret` 触发：**通过 4 次 `pop` 恢复占位的寄存器后，执行 `ret` 指令。`ret` 会将此时栈顶的线程函数入口地址弹出并载入 `EIP`（指令指针）。
- **结果：**CPU 认为某个函数刚刚“返回”到了目标位置，从而开始执行线程函数的第一条指令

2. 运行中的线程如何被换下与换上，并从中断点恢复？

被换下处理机（中断与保存）：

- **硬件介入：**线程正在执行时，时钟中断强行打断 CPU，硬件自动将当前的 EFLAGS、CS、EIP 压入当前线程的栈。
- **软件保存：**进入 `asm_switch_thread` 后，通过一系列 `push` 指令将 `ebp`、`ebx` 等通用寄存器压栈，并执行 `mov [eax], esp` 将当前的栈顶地址保存到 PCB 的 `stack` 成员中。此时，线程被中断那一刻的所有“状态”都被封存在了它自己的栈里。

换上并恢复执行（加载与跳转）：

- **恢复栈指针：**当调度器再次选中该线程，`asm_switch_thread` 执行 `mov esp, [eax]`。ESP 回到了该线程上次被换下时保存的那个位置。
- **弹出上下文：**通过 `pop` 指令恢复通用寄存器，执行 `ret` 回到 `schedule` 函数。
- **返回中断点：**随着 `c_time_interrupt_handler` 执行完毕，最终执行汇编级的 `iret`（或中断返回指令）。这条指令会将硬件当初压入的 EIP 重新弹回 CPU。
- **结果：**EIP 回到了线程当初被中断的那行代码地址，线程仿佛从未被中断过一样继续向下运行。

总结：操作系统如何实现并发执行？

操作系统的并发是通过“时间片轮转”和“上下文切换”实现的。

利用硬件中断周期性地强制暂停当前任务。利用 PCB（进程控制块）作为任务的“存档点”，将 CPU 的寄存器状态（上下文）保存到该任务私有的内核栈中。通过反复修改 CPU 的 ESP 和 EIP 寄存器，在不同的栈和指令流之间来回切换。

----- 实验任务 4 -----

- **任务要求：**在材料中，我们已经学习了如何使用时间片轮转算法来实现线程调度。但**线程调度算法**不止一种，例如先来先服务（最简单）最短作业（进程）优先 响应比最高优先算法 优先级调度算法 多级反馈队列调度算法 此外，我们的调度算法还可以是抢占式的。现在需要将线程调度算法修改为上

面提到的算法或者是自己设计的算法。然后， 同学们需要自行编写测试样例来呈现你的算法实现的正确性和基本逻辑。

- 思路分析：

1.我将选择**抢占式动态优先级调度算法**， 比较经典逻辑清晰且非常适合在已有的实验框架下实现

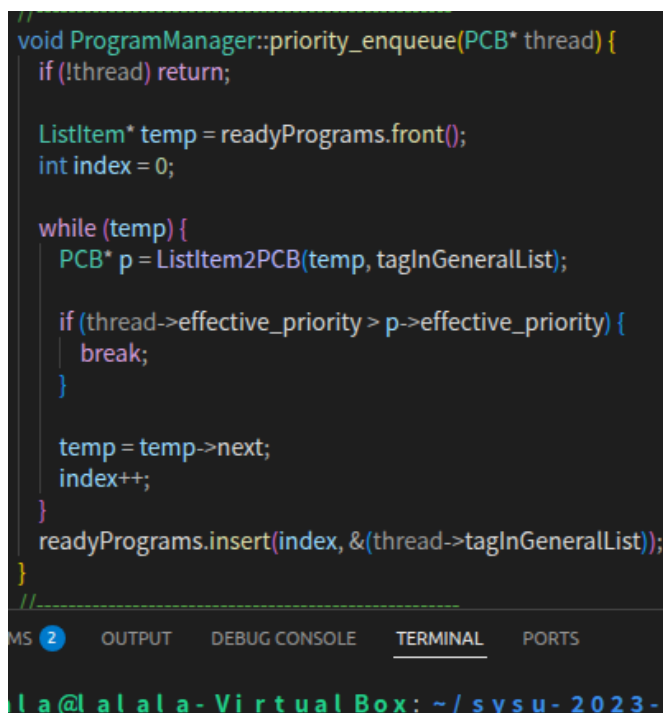
△算法核心思想：抢占式（如果就绪队列里有比当前线程优先级更高的线程，立即换下当前线程）+动态调整（如果线程用完了时间片，说明它是“计算密集型”，降低其优先级&&如果线程在就绪队列等得太久，可以提高其优先级）

2.注意类成员的声明： public or private

3.第一次输出通以优先级输出次数过多，难以观察到动态调整，于是采用调小时间片策略： ticks = priority * 10->running->ticks = 5;

- 实验步骤：

①声明以及实现辅助函数 `priority_enqueue(PCB* thread)`（注意结合 `list.cpp`）



```
//
void ProgramManager::priority_enqueue(PCB* thread) {
    if (!thread) return;

    ListItem* temp = readyPrograms.front();
    int index = 0;

    while (temp) {
        PCB* p = ListItem2PCB(temp, tagInGeneralList);

        if (thread->effective_priority > p->effective_priority) {
            break;
        }

        temp = temp->next;
        index++;
    }
    readyPrograms.insert(index, &(thread->tagInGeneralList));
}
//-----
MS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
la@lalala-VirtualBox: ~/sysu-2023-
```

②修改调度函数 `schedule`:

```
void ProgramManager::schedule()
{
    bool status = interruptManager.getInterruptStatus();
    interruptManager.disableInterrupt();

    if (readyPrograms.size() == 0)
    {
        interruptManager.setInterruptStatus(status);
        return;
    }

    if (running->status == ProgramStatus::RUNNING)
    {
        running->status = ProgramStatus::READY;
        // 动态优先级逻辑:
        // 如果线程是因为用光了时间片才被调度, 说明它很“贪婪”, 降低其动态优先级
        if (running->effective_priority > 1) {
            running->effective_priority -= 1;
        }

        // 根据新的动态优先级分配下一个时间片
        // running->ticks = running->effective_priority * 10;
        running->ticks = 5;
        priority_enqueue(running); // 按优先级插回队列
    }
    else if (running->status == ProgramStatus::DEAD)
    {
        releasePCB(running);
    }
}
```

MS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - build + v

ala@alala-VirtualBox: ~/sysu-2023-spring-operat

③修改 setup.cpp:

▲在 first_thread 中创建不同优先级的进程

```
programManager.executeThread(task_display, nullptr, "High_Pri", 10);

programManager.executeThread(task_display, nullptr, "Low_Pri", 2);
```

▲编写线程函数展现抢占式动态优先级调度算法的正确性

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学 2023 学年春季操作系统课程 - Gitee.com](#)